

## **Sequence Detector**

Using Mealy and Moore FSM — Detection of the Pattern “1010”

Verilog / SystemVerilog RTL Project Report

Overlapping and Non-Overlapping Detection

Language: SystemVerilog (.sv)

Waveform viewer: EPWave

**BHUMIKA U**

## Overview

This project implements a digital sequence detector that identifies the 4-bit pattern “1010” on a serial input, *din*. The detector is built four ways: using the Mealy FSM model and the Moore FSM model, and for each model, an overlapping and a non-overlapping detection policy. All four designs were simulated with dedicated testbenches and verified against their actual EPWave waveform captures.

## 1. Design (RTL)

Each design uses a two-block FSM coding style: a sequential always block updates the state register on the clock edge, and a combinational always block computes the next state (and, for Mealy, the output) from the present state and *din*.

### 1.1 Mealy FSM — state diagram

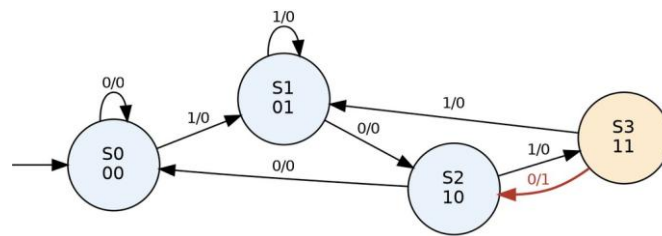


Fig. 1.1 — Mealy FSM (overlapping variant). The only difference in the non-overlapping variant is the  $S3 \rightarrow S2$  edge on  $din=0$ , which instead goes to  $S3 \rightarrow S0$ .

### 1.2 Moore FSM — state diagram

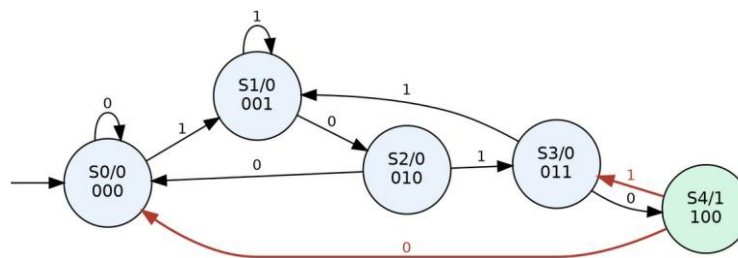


Fig. 1.2 — Moore FSM (overlapping variant, 5 states). The non-overlapping variant differs only in the  $S4$  edges:  $din=1$  goes to  $S1$  instead of  $S3$ .

### 1.3 State transition tables

**Mealy FSM (format: Next State / dout):**

| State   | Overlap din=0 | Overlap din=1 | Non-ov. din=0 | Non-ov. din=1 |
|---------|---------------|---------------|---------------|---------------|
| S0 (00) | S0 / 0        | S1 / 0        | S0 / 0        | S1 / 0        |
| S1 (01) | S2 / 0        | S1 / 0        | S2 / 0        | S1 / 0        |
| S2 (10) | S0 / 0        | S3 / 0        | S0 / 0        | S3 / 0        |
| S3 (11) | S2 / 1        | S1 / 0        | S0 / 1        | S1 / 0        |

**Moore FSM (dout depends on state only)**

| State    | dout | Ov. din=0 | Ov. din=1 | Non-ov. din=0 | Non-ov. din=1 |
|----------|------|-----------|-----------|---------------|---------------|
| S0 (000) | 0    | S0        | S1        | S0            | S1            |
| S1 (001) | 0    | S2        | S1        | S2            | S1            |
| S2 (010) | 0    | S0        | S3        | S0            | S3            |
| S3 (011) | 0    | S4        | S1        | S4            | S1            |
| S4 (100) | 1    | S0        | S3        | S0            | S1            |

## 2. Verilog RTL code

### Mealy — Overlapping (design.sv):

```

module seq_detector_1010_mealy(
    input clk,
    input rst,
    input din,
    output reg dout,
    output reg [1:0] state // 4 states fit perfectly into a 2-bit register bus
);
    // State Encoding
    parameter S0 = 2'b00, // Initial / Reset
               S1 = 2'b01, // Found "1"
               S2 = 2'b10, // Found "10"
               S3 = 2'b11; // Found "101"

    reg [1:0] next_state;

    // Sequential Block (State Register)
    always @(posedge clk or posedge rst) begin
        if (rst)
            state <= S0;
        else
            state <= next_state;
    end

    // Combinational Next State & Output Logic (Mealy style)
    always @(*) begin
        // Default values to prevent latches
        next_state = state;
        dout = 1'b0;

        case(state)
            S0: begin
                if (din) next_state = S1;
                else     next_state = S0;
            end

            S1: begin
                if (din) next_state = S1;
                else     next_state = S2;
            end

            S2: begin
                if (din) next_state = S3;
                else     next_state = S0;
            end

            S3: begin
                if (din) begin
                    next_state = S1;
                    dout = 1'b0;
                end else begin
                    // OVERLAPPING RULE:
                    // If din is 0 when in S3 (101), sequence "1010" is completed successfully.
                    // The last "10" overlaps to start the next sequence, so go back to S2.
                    next_state = S2;
                    dout = 1'b1; // Output fires IMMEDIATELY here
                end
            end

            default: next_state = S0;
        endcase
    end
endmodule

```

**Mealy — Non-overlapping (design.sv):**

```

module seq_detector_1010_mealy_non_overlap(
    input clk,
    input rst,
    input din,
    output reg dout,
    output reg [1:0] state
);
    // State Encoding
    parameter S0 = 2'b00, // Initial / Reset state
               S1 = 2'b01, // Found "1"
               S2 = 2'b10, // Found "10"
               S3 = 2'b11; // Found "101"

    reg [1:0] next_state;

    // Sequential State Register
    always @(posedge clk or posedge rst) begin
        if (rst)
            state <= S0;
        else
            state <= next_state;
    end

    // Combinational Next State & Output Logic
    always @(*) begin
        next_state = state;
        dout = 1'b0;

        case(state)
            S0: begin
                if (din) next_state = S1;
                else     next_state = S0;
            end

            S1: begin
                if (din) next_state = S1;
                else     next_state = S2;
            end

            S2: begin
                if (din) next_state = S3;
                else     next_state = S0;
            end

            S3: begin
                if (din) begin
                    next_state = S1; // Pattern broken: "1011" -> "1" can start a new sequence
                    dout = 1'b0;
                end else begin
                    // NON-OVERLAPPING RULE:
                    // Pattern "1010" completed!
                    // Fire the output immediately and reset back to S0.
                    next_state = S0;
                    dout = 1'b1;
                end
            end

            default: next_state = S0;
        endcase
    end
endmodule

```

**Moore — Overlapping (design.sv):**

```

module seq_detector_1010_overlap(
    input clk,
    input rst,
    input din,
    output reg dout,
    output reg [2:0] state
);
    // State Encoding
    parameter S0 = 3'b000, // Reset / Idle
               S1 = 3'b001, // Found "1"
               S2 = 3'b010, // Found "10"
               S3 = 3'b011, // Found "101"
               S4 = 3'b100; // Found "1010" (Success State)

    reg [2:0] next_state;

    // State Transition Register
    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else     state <= next_state;
    end

    // Next State Logic
    always @(*) begin
        case(state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: next_state = din ? S3 : S0;
            S3: next_state = din ? S1 : S4; // "1010" reached if din = 0

            // OVERLAPPING RULE:
            // If din = 1, pattern becomes ...10101 -> Last 3 bits are "101" (Go to S3)
            // If din = 0, pattern becomes ...10100 -> Resets back to start (Go to S0)
            S4: next_state = din ? S3 : S0;

            default: next_state = S0;
        endcase
    end

    // Moore Output assignment (Aligned to clock boundaries)
    always @(*) begin
        dout = (state == S4) ? 1'b1 : 1'b0;
    end
endmodule

```

**Moore — Non-overlapping (design.sv):**

```

module seq_detector_1010_overlap( input
    clk,
    input rst, input
    din, output reg
    dout,
    output reg [2:0] state
);
    // State Encoding
    parameter S0 = 3'b000, // Reset / Idle S1
               = 3'b001, // Found "1"
               S2 = 3'b010, // Found "10"
               S3 = 3'b011, // Found "101"
               S4 = 3'b100; // Found "1010" (Success State) reg

    [2:0] next_state;

    // State Transition Register
    always @(posedge clk or posedge rst) begin if
        (rst) state <= S0;
        else state <= next_state;
    end

    // Next State Logic always
    @(*) begin
        case(state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: next_state = din ? S3 : S0;
            S3: next_state = din ? S1 : S4; // "1010" reached if din = 0

            // OVERLAPPING RULE:
            // If din = 1, pattern becomes ...10101 -> Last 3 bits are "101" (Go to S3)
            // If din = 0, pattern becomes ...10100 -> Resets back to start (Go to S0)
            // S4: next_state = din ? S3 : S0;
            // NON-OVERLAPPING RULE:
            // Once "1010" is found (State 4), reset completely.
            // If din = 1, it counts as the first '1' of a brand new pattern (Go to S1).
            // If din = 0, it resets completely to the beginning (Go to S0). S4:
            next_state = din ? S1 : S0;

            default: next_state = S0;
        endcase
    end

    // Moore Output assignment (Aligned to clock boundaries)
    always @(*) begin
        dout = (state == S4) ? 1'b1 : 1'b0;
    end
endmodule

```

### 3. Testbench

Each testbench instantiates its design, generates a 10 ns clock, applies a reset pulse, then streams the bit sequence 1,0,1,0,1,0,0 into din — chosen so the pattern 1010 is matched twice if the detector is overlapping, and once if it is non-overlapping. All four testbenches dump their signals to dump.vcd for waveform viewing.

#### Mealy — Overlapping testbench:

```
`timescale 1ns / 1ps

module tb_seq_detector;
    reg clk;
    reg rst;
    reg din;
    wire dout;
    wire [1:0] state;

    // Instantiate Mealy FSM
    seq_detector_1010_mealy uut (
        .clk(clk),
        .rst(rst),
        .din(din),
        .dout(dout),
        .state(state)
    );

    // 10ns Clock generator
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_seq_detector);

        clk = 0; rst = 1; din = 0;
        #12 rst = 0; // Release reset out of sync with clock edge

        // --- Stream sequence: 1 -> 0 -> 1 -> 0 -> 1 -> 0 ---
        @(posedge clk); #1 din = 1; // Shifts to S1
        @(posedge clk); #1 din = 0; // Shifts to S2
        @(posedge clk); #1 din = 1; // Shifts to S3

        // Final bit of first 1010 sequence:
        @(posedge clk); #1 din = 0; // dout will go high during this clock cycle!

        // Overlapping verification bit:
        @(posedge clk); #1 din = 1; // Transitions from S2 to S3
        @(posedge clk); #1 din = 0; // dout goes high a second time!

        @(posedge clk); #1 din = 0; // Reset back to S0
        #40;
        $finish;
    end
endmodule
```

**Mealy — Non-overlapping testbench:**

```

`timescale 1ns / 1ps

module tb_seq_detector;
    reg clk;
    reg rst;
    reg din;
    wire dout;
    wire [1:0] state;

    // Instantiate Non-Overlapping Mealy FSM
    seq_detector_1010_mealy_non_overlap uut (
        .clk(clk),
        .rst(rst),
        .din(din),
        .dout(dout),
        .state(state)
    );
    // 10ns Clock generator
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_seq_detector);

        clk = 0; rst = 1; din = 0;
        #12 rst = 0; // Clear reset safely between clock cycles

        // --- Stream sequence: 1 -> 0 -> 1 -> 0 -> 1 -> 0 ---
        @(posedge clk); #1 din = 1; // Shifts to S1
        @(posedge clk); #1 din = 0; // Shifts to S2
        @(posedge clk); #1 din = 1; // Shifts to S3

        // Final bit of the sequence:
        @(posedge clk); #1 din = 0; // FIRST MATCH: dout pulses high immediately!

        // Overlapping bits are sent here:
        @(posedge clk); #1 din = 1; // Machine was reset to S0, so this din=1 shifts to S1
        @(posedge clk); #1 din = 0; // Shifts to S2

        @(posedge clk); #1 din = 0; // Stream breaks, back to S0
        #40;
        $finish;
    end
endmodule

```

**Moore — Overlapping testbench:**

```

`timescale 1ns / 1ps

module tb_seq_detector;
    reg clk;
    reg rst;
    reg din;
    wire dout;
    wire [2:0] state;

    // Connect to your design
    seq_detector_1010_overlap uut (
        .clk(clk),
        .rst(rst),
        .din(din),
        .dout(dout),
        .state(state)
    );

    // 10ns Clock cycle configuration
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_seq_detector);

        // Reset the system initially
        clk = 0;
        rst = 1;
        din = 0;
        #12;
        rst = 0; // Release reset cleanly between clock transitions

        // --- Stream sequence sequence: 1 -> 0 -> 1 -> 0 -> 1 -> 0 ---
        @(posedge clk); #1 din = 1; // Shifting to State 1
        @(posedge clk); #1 din = 0; // Shifting to State 2
        @(posedge clk); #1 din = 1; // Shifting to State 3
        @(posedge clk); #1 din = 0; // Hits State 4! (dout turns 1 on next cycle)

        // Testing Overlapping mechanics
        @(posedge clk); #1 din = 1; // Loops back to State 3 safely
        @(posedge clk); #1 din = 0; // Hits State 4 again!

        // Resetting sequence pattern
        @(posedge clk); #1 din = 0; // Back to State 0
        #40;
        $finish;
    end
endmodule

```

**Moore — Non-overlapping testbench:**

```

`timescale 1ns / 1ps

module tb_seq_detector;
    reg clk;
    reg rst;
    reg din;
    wire dout;
    wire [2:0] state;

    // Connect to your design
    seq_detector_1010_overlap uut (
        .clk(clk),
        .rst(rst),
        .din(din),
        .dout(dout),
        .state(state)
    );

    // 10ns Clock cycle configuration
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(1, tb_seq_detector);
        // Reset the system initially
        clk = 0;
        rst = 1;
        din = 0;
        #12;
        rst = 0; // Release reset cleanly between clock transitions

        // --- Stream sequence sequence: 1 -> 0 -> 1 -> 0 -> 1 -> 0 ---
        @(posedge clk); #1 din = 1; // Shifting to State 1
        @(posedge clk); #1 din = 0; // Shifting to State 2
        @(posedge clk); #1 din = 1; // Shifting to State 3
        @(posedge clk); #1 din = 0; // Hits State 4! (dout turns 1 on next cycle)

        // Testing Overlapping mechanics
        @(posedge clk); #1 din = 1; // Loops back to State 3 safely
        @(posedge clk); #1 din = 0; // Hits State 4 again!

        // Resetting sequence pattern
        @(posedge clk); #1 din = 0; // Back to State 0
        #40;
        $finish;
    end
endmodule

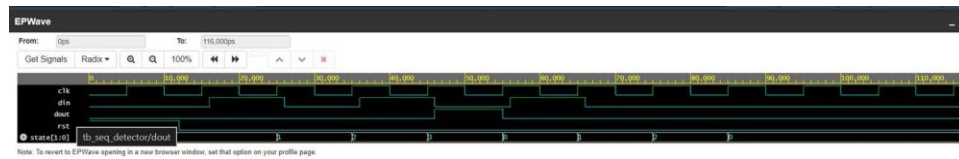
```

## 4. Simulation methodology

- Design and testbench compiled together for each of the four variants (design.sv + testbench.sv).
- Clock: 10 ns period, free-running from time 0 (always #5 clk = ~clk).
- Reset: rst held high for the first 12 ns, then released; state resets to S0.
- Stimulus: din driven with the fixed sequence 1,0,1,0,1,0,0, timed just after each clock edge.
- Waveform dump: \$dumpfile("dump.vcd") / \$dumpvars(1, tb\_seq\_detector), viewed in EPWave over 0–116,000 ps.

## 5. Waveforms

Unedited EPWave captures of the four simulation runs.



## 6. Results

| Design                  | State trace after first match | Matches (dout pulses) observed |
|-------------------------|-------------------------------|--------------------------------|
| Mealy — Non-overlapping | S3 → S0 → S1 → S2 → S0        | 1                              |
| Mealy — Overlapping     | S3 → S2 → S3 → S2 → S0        | 2                              |
| Moore — Non-overlapping | S4 → S1 → S2 → S0             | 1                              |
| Moore — Overlapping     | S4 → S3 → S4 → S0             | 2                              |

All four waveforms confirm correct behaviour: the overlapping designs detect two occurrences of 1010 in the test stream by reusing the trailing '10', while the non-overlapping designs detect one, discarding the match and returning to S0. Mealy's dout responds combinationaly (narrower pulse, same cycle as the input); Moore's dout is registered (a full clock-period level, one state-aligned pulse).

## 7. Conclusion

All four sequence-detector designs — Mealy and Moore, overlapping and non-overlapping — were implemented, simulated and verified against their actual waveforms. The results confirm correct pattern detection in every case, and clearly illustrate the practical differences between the Mealy and Moore models and between the two detection policies.